See attached instructions.

https://gemini.google.com/app/f162db429ce1d6b0

User prompt: See attached instructions.

Response: 1. CS1.1's Text: What Passes While the Claim Is False, and What Fails While It Holds

- **Agreed:** CS1.1 correctly fixes the population, the paired sign test on 100 markets, the nominal size of 0.043 without voids, the non-rejection naming rules, the unpowered region at 0.3 against 0.15–0.19, and the power loss from judging cut-off sessions that have at least one evaluation.
- **Disputed (1a — Passes while the claim is false: CS1.1's `Met` condition is ungated by the void-as-miss path and by Wall 5's reversed test):**
  - **Where:** `audit/program/briefs/1-positive-control.md`, Step 0:

    > "- **Met** when the upper boundary is crossed." "- *Passes while the claim is false:* … by a measure that pays something other than the drift captured, or pays a program that reads nothing. Walls 3 and 5 bound both before any session; by voids that hide losses… So the report also gives the decision with every void counted as the agent's miss, which holds whatever made the voids. A claim that this decision does not reach is reported with the sentence 'With its void sessions counted as misses, the claim is not reached'."

  - **Construction, with numbers:**

    1. *Void inflation:* Under CS1.1's text, the criterion is **Met** whenever the primary path crosses the upper boundary ($\ln 19$), even if the void-as-miss path does not. In Section 5.4, crossing the upper boundary prints the claim name and triggers the positive consequence (*"The second cell, a plant with no economic story, becomes worth its sessions"*), merely appending a caveat sentence. If a pre-evaluation failure channel correlates with earning (e.g., `claude-opus-5-5` vs `claude-opus-4-8` having different pre-evaluation crash rates and different earning rates), each selective void inflates the size of **Met** from 0.043 to 0.073 (1 void), 0.117 (2 voids), and 0.338 (5 voids).

    2. *Zero-search constant program passing Wall 5:* As proved in Point 3 below, Wall 5 (`CONSTANT_ALLOWED = 51` of 200, or 25.5%) tests the wrong null ($p_{\text{const}} \leq 0.18$). A constant position with true earning rate $p_{\text{const}} = 0.255$ passes Wall 5 with probability 0.526; an agent that reads nothing and submits that constant position on Turn 1 crosses CS1.1's upper boundary with probability 0.538 against G100 at 0.15 and 0.768 against G100 at 0.12.

- **Outcome it changes:** Allows CS1.1 to be formally **Met** at size > 0.05 when voids hide losses or when a constant position earns 0.18–0.255.
- **Severity: High.** Fix by requiring `CONSTANT_ALLOWED = 14` in Wall 5 (Point 3) and either gating **Met** on the void-as-miss path or keeping the runner active until the void-as-miss path resolves (Point 4).

- **Disputed (1b — Fails while the claim holds: the runner's stopping rule makes it algebraically impossible for the void-as-miss path to ever pass if any earning rerun had a void):**
  - **Where:** `audit/program/briefs/1-positive-control.md`, Step 8 item 6 and Step 9 item 4:

    > "6. Stop at `claim`, `futility` or `undecided`. With `continue`, go to market `i + 1`." "4. … the decision that the same markets give when every market with a void counts as the agent's miss, in place of its rerun's outcome."

  - **Construction, with numbers:** See Point 4 below for the exact proof. Because the runner stops on the *first* market K where the primary path reaches $S_K \geq \ln 19$, even a single innocent pre-evaluation network blip on any market $m \leq K$ where the rerun earns forces the secondary path's score at K strictly below $\ln 19$ with probability 1.000.
  - **Outcome it changes:** Turns a single innocent pre-evaluation network drop on an earning market into a 100%-guaranteed failure of the void-as-miss decision.
  - **Severity: Critical.**

2. The Agent as Served (Mixture of Two Models)

- **Agreed:** Reading `message.model` from assistant lines alone (ignoring `model_refusal_fallback` line order) is exact, and testing the composite served system (`claude-opus-5-5` with `claude-opus-4-8` fallback) preserves the unconditional sequential size of 0.043 whenever $\theta_m \leq 1/2$ holds for each market m and no voids occur.

- **Disputed (2a — Void-mediated model selection, non-stationary fallback drift, and endogenous mid-session fallback reporting):**
  - **Where:** `audit/program/architecture.md`, Section 5.2 and Section 5.4:

    > "The subject is the pilot's agent as it ran. The study is a control of the pilot's search, and the pilot's search was this mixture: every one of its results came from it." "The share of sessions answered by each id may drift during the study, and the size holds, because it is enumerated for every sequence of markets whose chance of a win is at most one half (5.2, between the arms)."

  - **Construction, with numbers:**

1. *Void-mediated model selection:* Suppose on Turn 1 before any evaluation, `M1` and `M2` are served 50/50 with earning rates $p_1 = 0.08, p_2 = 0.22$ (unconditional mixture rate $0.15 = p_{\mathrm{G100}}$, so the composite agent is no better than G100). If `M1` crashes before evaluation 1 on 50% of its launches (e.g., a hard safeguard refusal abort) while `M2` never crashes, rerunning voids shifts the counted mixture to $\frac{0.5}{0.5+0.25} = \frac{2}{3}$ `M2`, raising the counted agent's earning rate to $\frac{2}{3}(0.22) + \frac{1}{3}(0.08) = \mathbf{0.1733} > 0.15$ without any session ever seeing a score.

2. *Classifier drift between the pilot and Study 1:* The fallback share already drifted from $9/20 = 45\%$ in Stage 3 to $31/40 = 77.5\%$ in Stage 4. Suppose `claude-opus-5-5` earns at 0.45 and `claude-opus-4-8` at 0.05, with G100 at 0.15. Under Stage 4's 77.5% fallback, the pilot's mixture rate is $0.225(0.45) + 0.775(0.05) = \mathbf{0.140} < 0.15$ (the claim is false for the pilot's mixture). If the provider's live classifier flags only 10% of sessions during Study 1's week, Study 1's mixture rate is $0.90(0.45) + 0.10(0.05) = \mathbf{0.410}$, crossing `claim` with probability > 0.96. Similarly, if the classifier is relaxed on markets 1–25 ($\theta_m = 0.73$) and tightens on markets 26–100 ($\theta_m = 0.35$), the time-averaged mixture over the 100 markets has $\bar{\theta} = 0.445 < 0.5$, yet the early run of `claude-opus-5-5` sessions crosses the upper boundary by market 25 with probability 0.54.

3. *Endogenous mid-session fallback (`{claude-opus-5-5, claude-opus-4-8}`):* When `claude-opus-5-5` finds a strong program in 6 evaluations on an easy market, it stops early before triggering the safeguard; when a market is hard, it searches longer, increasing the chance of a mid-session fallback. Stratifying rates by model-ID set in the report (Step 9 item 3) without stating this survivorship bias makes `{5-5, 4-8}` look artificially weak relative to pure `{5-5}`.

- **Outcome it changes:** (1) Voids correlated with model ID inflate size unless the void-as-miss path gates the claim; (2) CS1.1's scope is the mixture *at Study 1's observed fallback share*, not the pilot's historical mixture share if the two diverge; (3) stratified model-set rates are confounded by stopping time.

- **Severity: Medium.** State in 5.4 / CS1.1 that the claim applies only at the fallback share observed during Study 1 and that mid-session fallback rates are confounded by search length; gate on the fixed void-as-miss path (Point 4).

3. Walls 3 and 5

- **Agreed:** Wall 3 (`EVEN_ALLOWED = 4` of 200 per mirror pair) settles D9-a: under $q \le 1/2$ ($p_0 = 0.003843$), the familywise false-stop probability across all 5 mirror pairs is $1 - (1 - \Pr(\mathrm{Binom}(200, 0.003843) > 4))^5 = 0.0052$, while a single mirror pair earning on 4.5% of markets is stopped 95.0% of the time.

- **Disputed (3a — Wall 5 tests the wrong null direction, letting a zero-search constant position pass Wall 5 and win CS1.1):**
  - **Where:** `audit/program/architecture.md`, Section 5.5, and `briefs/1-positive-control.md` (`CONSTANT_ALLOWED = 51`):

    > "5. **The plant is not free.** Constant long and constant short read nothing. Each earns on at most 51 of wall 3's 200 markets: a program at the top of G100's band, 0.18, exceeds that with probability 0.003. More means an agent could reach the claim with no search, by holding one side (Argon's round 5)."

  - **Construction, with numbers:**
    1. A constant position is a fixed deterministic program that reads nothing from market m; its score on `m/fresh/0..56` is independent of G100's search on m and has the same single-pair win probability q_{const} on every fresh pair of every market, giving market earning rate $p_{\text{const}} = \Pr(\text{Binom}(57, q_{\text{const}}) \geq 39)$.
    2. Because Wall 5 sets its cutoff at the upper 0.003 tail of $p = 0.18$ ($51/200 = \mathbf{0.255}$), a constant position with $q_{\text{const}} = 0.635$ ($p_{\text{const}} = 0.255$) **passes Wall 5 with probability** 0.526.
    3. An agent that performs zero search and always outputs that constant position ($p_A = 0.255$) against G100 at target $p_B = 0.15$ has $p_{\text{win}} = 0.255 \times 0.85 = 0.21675$ and $p_{\text{loss}} = 0.745 \times 0.15 = 0.11175$ ($\theta = 0.660$), crossing CS1.1's upper boundary (`claim`) with probability 0.538. Against G100 at $p_B = 0.12$ (inside the check band $[0.12, 0.18]$), $\theta = 0.715 > 0.70$ and it crosses `claim` with probability 0.768 (at $p_B = 0.11, 0.812$).
    4. Even at $p_{\text{const}} = 0.18$ (which passes Wall 5 with probability 0.997), against G100 at $p_B = 0.12$ ($\theta = 0.617$), the zero-search constant agent crosses `claim` with probability 0.284 ($6.6\times$ the nominal 0.043 size).
  - **Outcome it changes:** `CONSTANT_ALLOWED` in `constants.py`. Setting `CONSTANT_ALLOWED = 14` (7.0% of 200 markets) stops a constant position at the bottom of G100's band ($p_{\text{const}} = 0.11$) with probability $1 - \Pr(\text{Binom}(200, 0.11) \leq 14) = \mathbf{0.963}$ and at G100's target ($p_{\text{const}} = 0.15$) with probability 0.9996, while passing any constant position with $q_{\text{const}} \leq 0.55$ ($p_{\text{const}} \leq 0.0303$) with probability ≥ 0.9987 per side.
  - **Severity: High.**

4. The Void Rule and Its 0.073 and 0.117

- **Agreed:** Judging every session cut off after at least one evaluation on its kept ledger pick eliminates score-informed rerun bias, and Fable's backward induction (`0.073` at 1 void, `0.117` at 2 voids, `0.338` at 5 voids) is exact.

- **Disputed (4a — Stopping the runner when the primary path hits `claim` guarantees the void-as-miss path can NEVER reach `claim` whenever any earning rerun had a void):**

  - **Where:** `audit/program/briefs/1-positive-control.md`, Step 8 item 6 and Step 9 item 4:

    > "6. Stop at `claim`, `futility` or `undecided`. With `continue`, go to market `i + 1`." "4. Void sessions with their causes and model ids, and the decision that the same markets give when every market with a void counts as the agent's miss, in place of its rerun's outcome."

  - **Construction, with numbers:** Let $w = \ln(1.4) \approx 0.33647$, $\ell = \ln(0.6) \approx -0.51083$, and $U = \ln(19) \approx 2.94444$.

    1. The runner stops at the **first** market K where the primary path's log-likelihood ratio $S_K = W_K w + L_K \ell \geq U$. Because K is the first crossing, $S_k < U$ for all $k < K$, and $S_{K-1} = S_K - w < U$, so:

       $$S_K \in [U, U + w) = [2.94444, 3.28091)$$

    2. In the secondary path (`voids counted as misses`), every market m with a void has `agent_earns` set to `False`. Thus at every prefix $k \leq K$, $W'_k \leq W_k$ and $L'_k \geq L_k$, so $S'_k \leq S_k < U$ for all $k < K$: the secondary path can never cross before K.

    3. Now suppose **even one** market $m \leq K$ had a void before a rerun in which the agent earned (`agent_earns = True`):

       - If `searcher_earns = False` on m, market m was a discordant win `(True, False)` in the primary path and becomes `(False, False)` in the secondary path. Thus $W'_K \leq W_K - 1$ and $L'_K \geq L_K$, giving:

         $$S'_K \leq (W_K - 1)w + L_K \ell = S_K - w < (U + w) - w = U$$

       - If `searcher_earns = True` on m, market m was concordant `(True, True)` in the primary path and becomes a discordant loss `(False, True)` in the secondary path. Thus $W'_K \leq W_K$ and $L'_K \geq L_K + 1$, giving:

         $$S'_K \leq W_K w + (L_K + 1)\ell = S_K + \ell < U + w + \ell = U - 0.17435 < U$$

    4. Therefore, **if any market $m \leq K$ where the agent earned had a void, $S'_K < U$ holds with probability 1.000.** Because Step 8 item 6 stops the runner at K, the

secondary path is left stranded at `continue` (e.g., at 8 wins, 0 losses) and can **never** reach `claim`!

- **Outcome it changes:** Step 8 item 6 (`runner.py`). When the primary path reaches `claim` (locked in at K by Step 6 behaviour 5) while the void-as-miss path is still at `continue`, the runner must continue running markets until the void-as-miss path also leaves `continue` (or hits `MAX_MARKETS`).
- **Severity: Critical.**

5. The Stop on an Unnamed Model

- **Agreed:** Recording every `message.model` in `SERVED = ("claude-opus-5-5", "claude-opus-4-8")` and refusing to count sessions answered by an unapproved model prevents silent model substitution across resumed runs.
- **Disputed (5a — Checking `Model` before `Void` permanently kills the study on a single crashed 0-evaluation session whose synthetic error line was missed by the `ECONNREFUSED` probe):**
  - **Where:** `audit/program/architecture.md`, Section 5.2, and `briefs/1-positive-control.md`, Step 8 item 3 and behaviour 4:

    > "...made by running that build against an address that refuses connections, reading ids and line types only. The manifest names that id as the client's..." "3. **Model.** The session's model ids are read first, whatever else is true of the session... If any id is outside `SERVED`, the session is not counted... one line with the state 'model changed', the market's id and the ids is appended, and no session ever starts again under this manifest." "An id outside `SERVED` ends the study, in a session that reached a designed end and in one that crashed alike..."

  - **Construction, with numbers:**

    1. A raw TCP `ECONNREFUSED` probe only exercises transport-level connection failure. When an HTTP 500/502/529, stream drop, or auth/rate-limit wrapper occurs mid-request, the Claude Code CLI writes synthetic assistant error messages (typically with `message.model = "<synthetic>"`) that an `ECONNREFUSED` probe may not trigger.
    2. Because Step 8 item 3 evaluates `Model` *before* `Void`—even on a crashed session with **0 evaluations**—and makes `"model changed"` an irreversible terminal state (*"no session ever starts again under this manifest"*), a single transient API error on Market 15 that crashes before evaluation 1 with an unprobed client model string permanently bricks the entire manifest.
    3. Even for a genuine response from an unexpected model ID, the session is not counted and has seen no fresh pairs; pausing the runner (exactly as 3 voids do in

Step 8 item 2) costs 0.000 in Type I error while allowing `--end` if the provider permanently retired `SERVED` or resumption if the error was an account/routing blip.

- **Outcome it changes:** In Step 8 items 2–3 (`runner.py`), check `Void` (0 evaluations and no designed end) *before* triggering `"model changed"`, probe HTTP 500/529 (and existing transcripts on disk) when setting `NOTICE`, or make `"model changed"` pause the runner until `--end` is called.
- **Severity: Medium.**

Convergence Ledger (After Argon's Round 6)

Agreed

	Point	How it was agreed	Where
A35	Wall 3 checked per mirror pair at `EVEN_ALLOWED = 4` of 200 bounds the 5-pair familywise false-stop rate at 0.0052 and stops a 4.5% even-part pair 95% of the time. D9-a closes	Examined by Argon, round 6	5.5, Step 5
A36	Judging any session cut off after ≥ 1 evaluation on its kept ledger pick closes the score-informed rerun selection channel, and `voids()` (0.073 at 1 void, 0.117 at 2, 0.338 at 5) is exact	Examined by Argon, round 6	5.2, Step 8
A37	Reading a session's models from `message.model` on assistant lines alone (ignoring `model_refusal_fallback` order) is exact, and testing the composite served agent (`claude-opus-5-5`, `claude-opus-4-8`) preserves size 0.043 when $\theta_m \leq 1/2$ holds on each market without voids	Examined by Argon, round 6	5.1, 5.2, Step 8
A38	Steps 7–9 manifest hashing of the judged path (`verify()`) and storing each pick's inputs and G100's earlier bests in the JSONL lines so `report.py` reads no ledger	Examined by Argon, round 6	Steps 7–9

Disputed

	Dispute	Settled by	Changes
D9-b	**Wall 5's threshold (`CONSTANT_ALLOWED`).** The architect: 51 of 200 (25.5%, testing $p_{\text{const}} \leq 0.18$ at $\alpha = 0.003$). Argon: 51 of 200 tests the wrong direction—a	**A computation.** One constant in `constants.py` (`CONSTANT_ALLOWED = 14`)	**Yes,** `constants.py`

	Dispute	Settled by	Changes
	constant program at $p_{\text{const}} = 0.255$ passes Wall 5 52.6% of the time and beats G100 (0.15) 53.8% of the time (76.8% at 0.12) with zero search; set `CONSTANT_ALLOWED = 14` (7.0%), which rejects $p_{\text{const}} \geq 0.11$ at 96.3% (0.15 at 99.96%) and passes $q_{\text{const}} \leq 0.55$ at 99.9%		
D10-a	**The runner stops before the void-as-miss path can cross.** The architect: Step 8 stops as soon as the primary path hits `claim`, and Step 9 recomputes with voids as misses. Argon: because K is the first crossing ($S_K \in [\ln 19, \ln 19 + \ln 1.4)$), if even one market $m \leq K$ where the rerun earned had a void, $S'_K < \ln 19$ holds with probability 1.000, making it algebraically impossible for the void-as-miss path to ever reach `claim`	**An algebraic identity.** In Step 8 item 6, if the primary path hits `claim` while the void-as-miss path is at `continue`, keep running until the void-as-miss path also leaves `continue` (locking the primary decision at K per Step 6 behaviour 5)	**Yes,** `runner.py`, `test_runner.py`
D10-b	**CS1.1's `Met` condition and mixture drift note.** The architect: CS1.1 is `Met` on the primary path with a caveat sentence if the void-as-miss path fails. Argon: gate `Met` (or await the continued run from D10-a) on the void-as-miss path, and state in 5.4 that the claim holds for the fallback share observed during Study 1 and that stratified mid-session fallback rates are confounded by search duration	**Two sentences** in CS1.1 / 5.4	**Yes,** text of CS1.1 and 5.4
D16	**Unprobed error model strings killing the study on crashed 0-eval sessions.** The architect: probe `NOTICE` via `ECONNREFUSED` and check `Model` before `Void`, permanently ending the study on any other ID. Argon: check `Void` (0	**Ordering in Step 8 items 2–3** and the advisor's `NOTICE` check	**Yes,** `runner.py`, `test_runner.py`

	Dispute	Settled by	Changes
	evaluations, crashed) before `"model changed"`, also probe HTTP 500/529 and existing transcripts for `"<synthetic>"`, or pause rather than bricking the manifest		

D6, D7, D8 and Study 2's D11 to D15 stand as they were. Once D9-b, D10-a, D10-b, and D16 are applied, Study 1 and CS1.1 are **converged**.
